

SPECIAL PROBLEMS

Series Evaluation

A. $1 + 2 + 3 + \dots \dots \dots + N$ (can be done for any starting value and increment)

B. $1^3 + 2^3 + 3^3 + \dots \dots \dots + N^3$ (can be done for any power of n)

Solution A

```
#include <stdio.h>

int main()
{
    int N;
    printf("Enter the ending number of the series: ");
    scanf("%d",&N);
    int sum = 0; //initializaion of sum
    for(int i = 1; i<=N; i++)
    {
        sum = sum + i;
    }

    printf("%d",sum);

    return 0;
}
```

```
Enter the ending number of the series: 5
15

...Program finished with exit code 0
Press ENTER to exit console.
```

Solution B

```
#include <stdio.h>

int main()
{
    int N;
    printf("Enter the ending number of the series: ");
    scanf("%d",&N);
    int sum = 0; //initializaion of sum
    for(int i = 1; i<=N; i++)
    {
        sum = sum + i*i*i;
    }

    printf("%d",sum);

    return 0;
}
```

```
Enter the ending number of the series: 2
9

...Program finished with exit code 0
Press ENTER to exit console.
```

Series Evaluation

C. $1 \times 2 \times 3 \times \dots \times N$ (Factorial) (can be done for any starting value and increment)

D. $1^3 \times 2^3 \times 3^3 \times \dots \times N^3$ (can be done for any power of n)

Solution C

```
#include <stdio.h>

int main()
{
    int N;
    printf("Enter the ending number of the series: ");
    scanf("%d",&N);
    int prod = 1; //initializaion of product
    for(int i = 1; i<=N; i++)
    {
        prod = prod * i;
    }

    printf("%d",prod);

    return 0;
}
```

```
Enter the ending number of the series: 4
24

...Program finished with exit code 0
Press ENTER to exit console.
```

Solution D

```
#include <stdio.h>

int main()
{
    int N;
    printf("Enter the ending number of the series: ");
    scanf("%d",&N);
    int prod = 1; //initializaion of product
    for(int i = 1; i<=N; i++)
    {
        prod = prod *i*i*i;
    }

    printf("%d",prod);

    return 0;
}
```

```
Enter the ending number of the series: 2
8

...Program finished with exit code 0
Press ENTER to exit console.
```

Series Evaluation

$$\text{E. } \frac{1}{2} + \frac{1}{4} + \frac{1}{6} + \dots \dots \dots + \frac{1}{N}$$

$$\text{F. } \frac{1}{2^3} + \frac{1}{4^3} + \frac{1}{6^3} + \dots \dots \dots + \frac{1}{N^3}$$

$$\text{G. } 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{6} + \dots \dots \dots + \frac{1}{N}$$

$$\text{H. } \frac{1}{2!} + \frac{1}{4!} + \frac{1}{6!} + \dots \dots \dots + \frac{1}{N!}$$

Solution E

- Must declare as float

```
#include <stdio.h>

int main()
{
    int N;
    printf("Enter the ending number of the series: ");
    scanf("%d",&N);
    float sum = 0; //initializaion of sum
    for(float i = 2; i<=N; i = i+2)
    {
        sum = sum + (1/i);
    }

    printf("%.2f",sum);

    return 0;
}
```

```
Enter the ending number of the series: 4
0.75

...Program finished with exit code 0
Press ENTER to exit console.
```

Solution F

- Must declare as float and use proper parenthesis

```
#include <stdio.h>

int main()
{
    int N;
    printf("Enter the ending number of the series: ");
    scanf("%d",&N);
    float sum = 0; //initializaion of sum
    for(float i = 2; i<=N; i = i+2)
    {
        sum = sum + (1/(i*i*i));
    }

    printf("%.2f",sum);

    return 0;
}
```

```
Enter the ending number of the series: 4
0.14

...Program finished with exit code 0
Press ENTER to exit console.
```

Solution G

- Must declare as float and initialize sum as 1

```
#include <stdio.h>

int main()
{
    int N;
    printf("Enter the ending number of the series: ");
    scanf("%d",&N);
    float sum = 1; //initializaion of sum
    for(float i = 2; i<=N; i = i+2)
    {
        sum = sum + (1/i);
    }

    printf("%0.2f",sum);

    return 0;
}
```

```
Enter the ending number of the series: 4
1.75

...Program finished with exit code 0
Press ENTER to exit console.□
```

Solution H

- Must use nested loop to find the factorial and reset the fact variable every time

```
#include <stdio.h>

int main()
{
    int N;
    printf("Enter the ending number of the series: ");
    scanf("%d",&N);
    float sum = 0; //initializaion of sum
    for(int i = 2; i<=N; i = i + 2)
    {
        float fact = 1; //Declaring fact inside the loop to reset it at first
        for(float j =1; j<=i ; j++)
        {
            fact = fact * j; //finding 2! , 4! , 6! and so on
        }
        sum = sum + (1/fact); // finding sum = 1/2! + 1/4! + 1/6! and so on
    }

    printf("%.2f",sum);

    return 0;
}
```

```
Enter the ending number of the series: 6
0.54

...Program finished with exit code 0
Press ENTER to exit console.
```

Series Evaluation

I. $1 + 1 \times 2 + 1 \times 2 \times 3 + \dots \dots \dots + 1 \times 2 \times 3 \times \dots \dots \dots \times N$

$\Rightarrow$ Can also be written as $1! + 2! + 3! + \dots \dots \dots + N!$

J. $1 + 2^2 + 3^3 + 4^4 + \dots \dots \dots + N^N$

K. $\frac{1 \times 2}{3!} + \frac{2 \times 3}{4!} + \frac{3 \times 4}{5!} + \dots \dots \dots$ (use two loops)

Solution I

```
#include <stdio.h>

int main()
{
    int N;
    printf("Enter the ending number of the series: ");
    scanf("%d",&N);
    int sum = 1; //initializaion of sum
    for(int i = 2; i<=N; i++)
    {
        int fact = 1; //Declaring fact inside the loop to reset it at first
        for(int j =1; j<=i ; j++)
        {
            fact = fact * j; //finding 2! , 3! and so on
        }
        sum = sum + fact; // finding sum = 1 + 2! + 3! and so on
    }

    printf("%d",sum);

    return 0;
}
```

```
Enter the ending number of the series: 4
33
...Program finished with exit code 0
Press ENTER to exit console.
```

Solution J

```
#include <stdio.h>
#include<math.h>

int main()
{
    int n;
    printf("Enter the ending number of the series: ");
    scanf("%d",&n);

    int sum = 0;

    int i;
    for(i=1 ; i<=n ;i++)
    {
        sum = sum + pow(i,i);
    }

    printf("%d",sum);

    return 0;
}
```

```
Enter the ending number of the series: 2
5
...Program finished with exit code 0
Press ENTER to exit console.
```

Solution K

```
#include <stdio.h>

int main()
{
    int n;
    printf("Enter the ending number of the series: ");
    scanf("%d",&n);

    float sum = 0;

    float i;
    for(i=1 ; i<=n ; i++)
    {
        float numerator = i*(i+1);

        float fact =1;
        float j;
        for(j=1 ; j<= i+2 ; j++)
        {
            fact = fact * j;
        }

        float denominator = fact;

        float fraction = numerator/denominator;

        sum = sum + fraction;
    }

    printf("%f",sum);

    return 0;
}
```

```
Enter the ending number of the series: 3
0.683333
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```


RECORDING FLIPPED MODE

Fibonacci Series

In mathematics, the Fibonacci sequence is a sequence in which each number is the sum of the two preceding ones.

0, 1, 1, 2, 3, 5, 8, 13, 21

0 1 1 2 3 5 8



term1

term2

Fibonacci Series

```
#include <stdio.h>
int main() {

    int i, n;

    // initialize first and second terms
    int t1 = 0, t2 = 1;

    // initialize the next term (3rd term)
    int nextTerm = t1 + t2;

    printf("Enter the number of terms: ");
    scanf("%d", &n);

    // print the first two terms t1 and t2
    printf("Fibonacci Series: %d, %d, ", t1, t2);

    // print 3rd to nth terms
    for (i = 3; i <= n; ++i) {
        printf("%d, ", nextTerm);
        t1 = t2;
        t2 = nextTerm;
        nextTerm = t1 + t2;
    }

    return 0;
}
```

```
Enter the number of terms: 8
Fibonacci Series: 0, 1, 1, 2, 3, 5, 8, 13,
Process returned 0 (0x0)   execution time : 2.982 s
Press any key to continue.
```

Lucas Series

The Lucas series is a sequence of numbers, named after mathematician Édouard Lucas, that follows the same rule as the Fibonacci sequence (each number is the sum of the two preceding ones) but starts with different initial terms: 2, 1 instead of 0, 1.

Individual digit extraction (will be very useful in embedded system ADC)

```
#include <stdio.h>

int main()
{
    int num = 5678;

    int d_thousands = (num/1000)%10;
    int d_hundreds = (num/100)%10;
    int d_tens = (num/10)%10;
    int d_ones = (num/1)%10;

    printf("Digit ones = %d\n",d_ones);
    printf("Digit tens = %d\n",d_tens);
    printf("Digit hundreds = %d\n",d_hundreds);
    printf("Digit thousands = %d\n",d_thousands);

    return 0;
}
```

```
Digit ones = 8
Digit tens = 7
Digit hundreds = 6
Digit thousands = 5
```

```
=== Code Execution Successful ===
```

Individual digit extraction and its sum (using loops)

```
#include<stdio.h>

int main()
{
    int num, sum =0;

    printf("Enter a number to find the sum of its digits: ");
    scanf("%d",&num);

    while(num != 0)
    {
        int individualDigit = num %10;
        printf("\nThe individual digit is %d\n",individualDigit);
        sum = sum + individualDigit;
        num = num/10;
    }

    printf("\nThe sum is %d\n",sum);

    return 0;
}
```

```
Enter a number to find the sum of its digits: 512

The individual digit is 2
The individual digit is 1
The individual digit is 5

The sum is 8

Process returned 0 (0x0)   execution time : 1.664 s
Press any key to continue.
```

Armstrong Number

An Armstrong number is a number equal to the sum of its own digits, each raised to the power of the total number of digits.

For example, if you have a 3-digit number like 153, it's an Armstrong number because $1^3 + 5^3 + 3^3$ equals 153.

Similarly, if you have a 4-digit number like 1634, it's an Armstrong number because $1^4 + 6^4 + 3^4 + 4^4$ equals 1634

Armstrong Number (contd.)

```
#include <stdio.h>
#include <math.h>

int main() {
    int num, num1, originalNum, count = 0, individualDigit, sum = 0;

    printf("Enter an integer: ");
    scanf("%d", &num);

    originalNum = num;

    // Count number of digits
    while (num != 0) {
        num = num/10;
        count++;
    }

    num1 = originalNum;

    // Calculate the sum of digits raised to power n
    while (num1 != 0) {
        individualDigit = num1 % 10;
        sum = sum + round(pow(individualDigit, count));
        num1 = num1/10;
    }

    // Check Armstrong condition
    if (originalNum == sum)
        printf("%d is an Armstrong number.\n", originalNum);
    else
        printf("%d is not an Armstrong number.\n", originalNum);

    return 0;
}
```

```
Enter an integer: 153
153 is an Armstrong number.
```

```
Process returned 0 (0x0)   execution time : 1.548 s
Press any key to continue.
```


Palindrome

A palindrome number reads the same forwards and backward, such as 121 or 8008.

Palindrome

```
#include <stdio.h>

int main()
{
    int num, originalNum, reversedNum = 0;

    printf("Enter a number to check palindrome: ");
    scanf("%d", &num);

    originalNum = num;

    while (num != 0)
    {
        int digit = num % 10; // extract last digit
        reversedNum = reversedNum * 10 + digit; // build reversed number
        num = num / 10;
    }

    if (reversedNum == originalNum)
        printf("\n%d is a Palindrome number.\n", originalNum);
    else
        printf("\n%d is NOT a Palindrome number.\n", originalNum);

    return 0;
}
```

Enter a number to check palindrome: 313

313 is a Palindrome number.

Process returned 0 (0x0) execution time : 1.970 s
Press any key to continue.

Reverse of 123 using Iteration (Short)

Initial values:

num = 123, reversedNum = 0

• Iteration 1:

digit = $123 \% 10 = 3$

reversedNum = $0 \times 10 + 3 = 3$

num = $123 / 10 = 12$

• Iteration 2:

digit = $12 \% 10 = 2$

reversedNum = $3 \times 10 + 2 = 32$

num = $12 / 10 = 1$

• Iteration 3:

digit = $1 \% 10 = 1$

reversedNum = $32 \times 10 + 1 = 321$

num = $1 / 10 = 0$ (stop)

Final reversed number = 321

Strong Number

A strong number is a number where the sum of the factorials of its individual digits equals the number itself. For example, 145 is a strong number because $1! + 4! + 5! = 145$.

Strong Number

```
#include <stdio.h>

int main()
{
    int num, originalNum, sum = 0;

    printf("Enter a number to check Strong number: ");
    scanf("%d", &num);

    originalNum = num;

    while (num != 0)
    {
        int digit = num % 10;

        int fact = 1;
        for (int i = 1; i <= digit; i++)
        {
            fact = fact * i;
        }

        sum = sum + fact;
        num = num / 10;
    }

    if (sum == originalNum)
        printf("\n%d is a Strong Number.\n", originalNum);
    else
        printf("\n%d is NOT a Strong Number.\n", originalNum);

    return 0;
}
```

Enter a number to check Strong number: 145

145 is a Strong Number.

Process returned 0 (0x0) execution time : 6.622 s
Press any key to continue.